

Code Evidence: Finite-Volume Solver and Tensor-Network Compression Module

This document contains complete, verbatim source listings of the two core modules backing the submission's claims. They are provided as PDF because the portal did not accept .py uploads. The full repository (solver, octonion algebra, tests, benchmark scripts) reproduces every reported number; reproduction commands are in the main report. All code is standard scientific Python (NumPy, TensorLy).

Part A - finite_volume_solver.py

The working 2D Convecting Taylor-Green Vortex solver: cell-centered periodic finite volume, face fluxes, RK2 stepping, FFT (spectral) pressure projection. Verbatim copy of src/airbus_tgv/finite_volume.py.

```

1  # =====
2  # SUBMISSION EVIDENCE - Airbus 2026 Quantum + AI Challenge, Phase 1
3  # Verbatim copy of src/airbus_tgv/finite_volume.py from the project repository.
4  # The 2D Convecting Taylor-Green Vortex finite-volume solver: cell-centered
5  # periodic FV, face fluxes, RK2 stepping, FFT (spectral) pressure projection.
6  # Relative imports (.constants, .exact_solution, .fluxes) refer to sibling
7  # modules in the airbus_tgv package; full reproduction commands are in the
8  # accompanying PDF report, Section 9.
9  # =====
10
11 """Cell-centered, periodic finite-volume solver for the 2D incompressible
12 Navier-Stokes equations using a fractional-step projection method.
13
14 State variables: cell-averaged velocities u, v on an nx x ny grid covering
15 [0, 2*pi]^2 with periodic BCs. Pressure is computed each step via an FFT
16 Poisson solve to enforce incompressibility.
17 """
18 from __future__ import annotations
19 from dataclasses import dataclass, field
20 from typing import Callable, Optional
21 import numpy as np
22
23 from .constants import TGVParams
24 from .exact_solution import exact_velocity, divergence as div_centered
25 from .fluxes import (
26     convective_fluxes, diffusive_fluxes, divergence_from_faces,
27     face_average_x, face_average_y,
28 )
29
30
31 def build_periodic_grid(nx: int, ny: int, L_domain: float = 2.0 * np.pi):
32     dx = L_domain / nx
33     dy = L_domain / ny
34     x = (np.arange(nx) + 0.5) * dx
35     y = (np.arange(ny) + 0.5) * dy
36     X, Y = np.meshgrid(x, y, indexing="ij")
37     return X, Y, dx, dy
38
39
40 def initialize_cell_averages_from_exact(nx: int, ny: int, params: TGVParams):
41     L_domain = 2.0 * np.pi
42     X, Y, dx, dy = build_periodic_grid(nx, ny, L_domain)
43     u, v = exact_velocity(X, Y, 0.0, params)
44     return X, Y, dx, dy, u, v
45
46
47 def poisson_solve_periodic(rhs: np.ndarray, dx: float, dy: float) -> np.ndarray:
48     nx, ny = rhs.shape
49     kx = 2.0 * np.pi * np.fft.fftfreq(nx, d=dx)
50     ky = 2.0 * np.pi * np.fft.fftfreq(ny, d=dy)
51     KX, KY = np.meshgrid(kx, ky, indexing="ij")
52     K2 = KX * KX + KY * KY
53     K2[0, 0] = 1.0
54     rhs_hat = np.fft.fftn(rhs - rhs.mean())
55     phi_hat = -rhs_hat / K2
56     phi_hat[0, 0] = 0.0
57     return np.real(np.fft.ifftn(phi_hat))
58
59

```

```

60 def project_divergence_free(u, v, dx, dy):
61     nx, ny = u.shape
62     kx = 2.0 * np.pi * np.fft.fftfreq(nx, d=dx)
63     ky = 2.0 * np.pi * np.fft.fftfreq(ny, d=dy)
64     KX, KY = np.meshgrid(kx, ky, indexing="ij")
65     K2 = KX * KX + KY * KY
66     K2[0, 0] = 1.0
67     u_hat = np.fft.fftn(u)
68     v_hat = np.fft.fftn(v)
69     div_hat = 1j * (KX * u_hat + KY * v_hat)
70     phi_hat = -div_hat / K2
71     phi_hat[0, 0] = 0.0
72     u_proj_hat = u_hat - 1j * KX * phi_hat
73     v_proj_hat = v_hat - 1j * KY * phi_hat
74     phi = np.real(np.fft.ifftn(phi_hat))
75     return np.real(np.fft.ifftn(u_proj_hat)), np.real(np.fft.ifftn(v_proj_hat)), phi
76
77
78 def compute_face_states(u, v):
79     """Return arithmetic face states for x/y faces from cell averages."""
80     return {
81         "u_x": face_average_x(u),
82         "v_x": face_average_x(v),
83         "u_y": face_average_y(u),
84         "v_y": face_average_y(v),
85     }
86
87
88 def compute_convective_fluxes(u, v):
89     """Public wrapper for conservative momentum fluxes across volume faces."""
90     return convective_fluxes(u, v, 0.0, 0.0)
91
92
93 def compute_diffusive_fluxes(u, v, nu, dx, dy):
94     """Public wrapper for viscous face fluxes."""
95     return diffusive_fluxes(u, v, nu, dx, dy)
96
97
98 def rhs_fv(u, v, nu, dx, dy, lf_coeff: float = 1.0):
99     Fx_u, Fx_v, Fy_u, Fy_v = convective_fluxes(u, v, 0.0, 0.0, lf_coeff=lf_coeff)
100    Gx_u, Gx_v, Gy_u, Gy_v = diffusive_fluxes(u, v, nu, dx, dy)
101    dudt = -divergence_from_faces(Fx_u - Gx_u, Fy_u - Gy_u, dx, dy)
102    dvdt = -divergence_from_faces(Fx_v - Gx_v, Fy_v - Gy_v, dx, dy)
103    return dudt, dvdt
104
105
106 def cfl_dt(u, v, dx, dy, nu, cfl: float = 0.4, vis_cfl: float = 0.2) -> float:
107     umax = float(np.max(np.abs(u))) + 1e-12
108     vmax = float(np.max(np.abs(v))) + 1e-12
109     dt_adv = cfl / (umax / dx + vmax / dy)
110     dt_vis = vis_cfl * 0.5 / (nu * (1.0 / dx ** 2 + 1.0 / dy ** 2) + 1e-30)
111     return float(min(dt_adv, dt_vis))
112
113
114 def step_finite_volume(u, v, dt, nu, dx, dy, project: bool = True,
115                       lf_coeff: float = 1.0):
116     du1, dv1 = rhs_fv(u, v, nu, dx, dy, lf_coeff=lf_coeff)
117     u_mid = u + 0.5 * dt * du1
118     v_mid = v + 0.5 * dt * dv1
119     du2, dv2 = rhs_fv(u_mid, v_mid, nu, dx, dy, lf_coeff=lf_coeff)
120     u_new = u + dt * du2
121     v_new = v + dt * dv2
122     if project:
123         u_new, v_new, _ = project_divergence_free(u_new, v_new, dx, dy)
124     return u_new, v_new
125
126
127 @dataclass
128 class FVRunResult:
129     u: np.ndarray
130     v: np.ndarray
131     t: float
132     n_steps: int
133     dt_history: list = field(default_factory=list)
134     diagnostics: dict = field(default_factory=dict)
135
136

```

```

137 def run_fv_solver(params: TGVParams,
138                  nx: int = 64, ny: int = 64,
139                  t_final: float = 1.0,
140                  cfl: float = 0.4,
141                  project: bool = True,
142                  lf_coeff: float = 1.0,
143                  dt_controller: Optional[Callable] = None,
144                  callback: Optional[Callable] = None,
145                  initial_condition: Optional[Callable] = None,
146                  max_steps: int = 200_000) -> FVRunResult:
147     """initial_condition, if given, is a callable (X, Y, params) -> (u, v)
148     replacing the default exact-TGV initialization."""
149     X, Y, dx, dy, u, v = initialize_cell_averages_from_exact(nx, ny, params)
150     if initial_condition is not None:
151         u, v = initial_condition(X, Y, params)
152     if project:
153         u, v, _ = project_divergence_free(u, v, dx, dy)
154     nu = params.nu
155     t = 0.0
156     dt_hist = []
157     steering_factors = []
158     diag = {"unstable": False, "reason": None}
159     for step in range(max_steps):
160         if t >= t_final - 1e-15:
161             break
162         dt = cfl_dt(u, v, dx, dy, nu, cfl=cfl)
163         if dt_controller is not None:
164             factor = float(dt_controller(u, v, dx, dy, step, t))
165             if not np.isfinite(factor) or factor <= 0.0 or factor > 1.0:
166                 raise ValueError(f"dt_controller returned invalid factor {factor}")
167             dt *= factor
168             steering_factors.append(factor)
169         if t + dt > t_final:
170             dt = t_final - t
171         u, v = step_finite_volume(u, v, dt, nu, dx, dy, project=project,
172                                 lf_coeff=lf_coeff)
173         t += dt
174         dt_hist.append(dt)
175         if not np.isfinite(u).all() or not np.isfinite(v).all():
176             diag["unstable"] = True
177             diag["reason"] = f"NaN/Inf at step {step}"
178             break
179         if callback is not None:
180             callback(step, t, u, v)
181     diag.update({"X": X, "Y": Y, "dx": dx, "dy": dy,
182               "dt_steering_factors": steering_factors})
183     return FVRunResult(u=u, v=v, t=t, n_steps=len(dt_hist),
184                       dt_history=dt_hist, diagnostics=diag)

```

Part B - tensor_network_compression.py

The Tensor Network within Finite Volume core: space-time (time, x, y) Tucker decomposition with octonion-associator-guided rank selection. This module produced the measured compression results in the report. Verbatim copy of src/airbus_tgv/trajectory.py.

```

1  # =====
2  # SUBMISSION EVIDENCE - Airbus 2026 Quantum + AI Challenge, Phase 1
3  # Verbatim copy of src/airbus_tgv/trajectory.py from the project repository.
4  # This is the Tensor Network within Finite Volume core: the full space-time
5  # (time, x, y) solution tensor is Tucker-decomposed (TensorLy), with per-mode
6  # ranks selected automatically from octonion-associator torsion statistics.
7  # This module produced the measured 1010x compression at 1.4e-15 error on the
8  # 256^2 grid reported in the PDF (Section 4). Relative imports refer to
9  # sibling modules in the airbus_tgv package.
10 # =====
11
12 """Trajectory snapshot collection and space-time tensor operations.
13
14 Collect full (time, x, y) snapshots during a simulation, then decompose
15 into a low-rank Tucker/TT tensor where rank is guided by associator
16 field diagnostics.
17 """
18 from __future__ import annotations
19 from dataclasses import dataclass, field
20 from typing import Optional
21 import numpy as np
22

```

```

23 from .constants import TGVParams
24 from .finite_volume import run_fv_solver
25 from .exact_solution import exact_velocity, kinetic_energy
26 from .associator_metrics import associator_field, associator_summary_stats
27 from .compression import memory_estimate_dense
28
29 try:
30     import tensorly as tl # type: ignore
31     from tensorly.decomposition import tucker # type: ignore
32     _HAS_TENSORLY = True
33 except Exception:
34     _HAS_TENSORLY = False
35
36
37 @dataclass
38 class Trajectory:
39     """Collected snapshots: list of (u, v, t) tuples."""
40     times: list = field(default_factory=list)
41     u_snapshots: list = field(default_factory=list)
42     v_snapshots: list = field(default_factory=list)
43     params: Optional[TGVParams] = None
44     X: Optional[np.ndarray] = None
45     Y: Optional[np.ndarray] = None
46     dx: float = 0.0
47     dy: float = 0.0
48
49     def add_snapshot(self, u, v, t):
50         """Append a snapshot."""
51         self.times.append(t)
52         self.u_snapshots.append(u.copy())
53         self.v_snapshots.append(v.copy())
54
55     def to_tensor(self) -> tuple:
56         """Stack snapshots into (nt, nx, ny) tensors."""
57         if not self.times:
58             raise ValueError("No snapshots collected")
59         U = np.stack(self.u_snapshots, axis=0) # (nt, nx, ny)
60         V = np.stack(self.v_snapshots, axis=0)
61         return U, V
62
63     def memory_dense_mb(self) -> float:
64         """Dense memory footprint of the full trajectory."""
65         if not self.u_snapshots:
66             return 0.0
67         U, V = self.to_tensor()
68         return memory_estimate_dense(U) + memory_estimate_dense(V)
69
70
71 def collect_trajectory(params: TGVParams, nx: int = 64, ny: int = 64,
72                       t_final: float = 0.5, cfl: float = 0.4,
73                       snapshot_interval: int = 1,
74                       lf_coeff: float = 0.0,
75                       initial_condition=None) -> Trajectory:
76     """Run FV solver and collect snapshots at regular intervals.
77
78     lf_coeff=0.0 uses the 2nd-order central flux (default; matches the
79     benchmark path), 1.0 restores full Lax-Friedrichs dissipation.
80     initial_condition, if given, is passed through to run_fv_solver.
81     """
82     traj = Trajectory(params=params)
83     step_count = [0]
84
85     def snapshot_callback(step, t, u, v):
86         if step_count[0] % snapshot_interval == 0:
87             traj.add_snapshot(u, v, t)
88             step_count[0] += 1
89
90     res = run_fv_solver(params, nx=nx, ny=ny, t_final=t_final, cfl=cfl,
91                       lf_coeff=lf_coeff, callback=snapshot_callback,
92                       initial_condition=initial_condition)
93
94     # Store geometry
95     traj.X = res.diagnostics["X"]
96     traj.Y = res.diagnostics["Y"]
97     traj.dx = res.diagnostics["dx"]
98     traj.dy = res.diagnostics["dy"]
99

```

```

100     # Ensure final snapshot is included
101     if not traj.times or abs(traj.times[-1] - res.t) > 1e-10:
102         traj.add_snapshot(res.u, res.v, res.t)
103
104     return traj
105
106
107 def associator_guided_ranks(traj: Trajectory, min_rank: int = 4,
108                             max_rank: int = 32) -> dict:
109     """Compute per-mode ranks for Tucker decomposition guided by associator.
110
111     Collapse each mode of the trajectory tensor and compute associator
112     statistics along that mode. Higher associator torsion => keep more rank.
113     """
114     U, V = traj.to_tensor()
115     nt, nx, ny = U.shape
116
117     ranks = {"time": min_rank, "x": min_rank, "y": min_rank}
118
119     if traj.X is None or traj.Y is None:
120         return ranks
121
122     # Compute associator at a few key timepoints
123     A_stats = []
124     for i in [0, len(traj.times) // 2, len(traj.times) - 1]:
125         A_dict = associator_field(U[i], V[i], None, traj.dx, traj.dy, mode="strain")
126         stats = associator_summary_stats(A_dict["A"])
127         A_stats.append(stats["associator_p95"])
128
129     # Rank allocation: use p95 associator to guide
130     A_mean = np.mean(A_stats)
131     A_max = np.max(A_stats)
132     if A_max > 0:
133         ratio = A_mean / A_max
134     else:
135         ratio = 0.5
136
137     # Scale ranks based on associator magnitude
138     base_rank = int(min_rank + ratio * (max_rank - min_rank))
139     base_rank = max(min_rank, min(max_rank, base_rank))
140
141     # Allocate ranks proportional to mode size, clamped so the compressed
142     # representation is guaranteed to be smaller than the dense tensor:
143     # never exceed half the mode dimension (and never exceed the dimension).
144     ranks["time"] = _clamp_rank(int(0.5 * base_rank), min_rank, max_rank, nt)
145     ranks["x"] = _clamp_rank(base_rank, min_rank, max_rank, nx)
146     ranks["y"] = _clamp_rank(base_rank, min_rank, max_rank, ny)
147
148     return ranks
149
150
151 def _clamp_rank(rank: int, min_rank: int, max_rank: int, dim: int) -> int:
152     """Clamp a requested rank to [min_rank, max_rank], then cap at dim//2
153     (never below 1) so compression is guaranteed to shrink the mode."""
154     rank = max(min_rank, min(rank, max_rank))
155     return max(1, min(rank, dim // 2))
156
157
158 def tucker_decompose_trajectory(traj: Trajectory, core_ranks: Optional[dict] = None) -> dict:
159     """Decompose trajectory tensor U and V using Tucker decomposition.
160
161     Returns dict with 'U', 'V', 'core_u', 'core_v', 'factors_u', 'factors_v',
162     'ranks', 'compression_ratio', 'reconstruction_error'.
163     """
164     if not _HAS_TENSORLY:
165         raise RuntimeError(
166             "tensorly is required for Tucker decomposition. "
167             "Install with: pip install tensorly"
168         )
169
170     U, V = traj.to_tensor()
171
172     if core_ranks is None:
173         core_ranks = associator_guided_ranks(traj)
174
175     nt, nx, ny = U.shape
176     # Never let a rank exceed its mode dimension (tensorly would error or

```

```

177     # the "compressed" form would be larger than the dense tensor).
178     ranks = [max(1, min(core_ranks["time"], nt)),
179              max(1, min(core_ranks["x"], nx)),
180              max(1, min(core_ranks["y"], ny))]
181
182     U_tensor = tl.tensor(U)
183     V_tensor = tl.tensor(V)
184
185     # Tucker decomposition
186     core_u, factors_u = tucker(U_tensor, rank=ranks)
187     core_v, factors_v = tucker(V_tensor, rank=ranks)
188
189     # Reconstruction
190     U_rec = tl.tucker_to_tensor((core_u, factors_u))
191     V_rec = tl.tucker_to_tensor((core_v, factors_v))
192
193     U_rec_np = np.array(U_rec)
194     V_rec_np = np.array(V_rec)
195
196     # Error
197     U_error = float(np.linalg.norm(U - U_rec_np)) / (np.linalg.norm(U) + 1e-30)
198     V_error = float(np.linalg.norm(V - V_rec_np)) / (np.linalg.norm(V) + 1e-30)
199     mean_error = 0.5 * (U_error + V_error)
200
201     # Memory
202     mem_dense = traj.memory_dense_mb()
203     mem_core_u = core_u.nbytes / (1024.0 * 1024.0)
204     mem_core_v = core_v.nbytes / (1024.0 * 1024.0)
205     mem_factors = sum(f.nbytes for f in factors_u + factors_v) / (1024.0 * 1024.0)
206     mem_compressed = mem_core_u + mem_core_v + mem_factors
207
208     return {
209         "U": U,
210         "V": V,
211         "core_u": core_u,
212         "core_v": core_v,
213         "factors_u": factors_u,
214         "factors_v": factors_v,
215         "U_reconstructed": U_rec_np,
216         "V_reconstructed": V_rec_np,
217         "ranks": ranks,
218         "compression_ratio": mem_dense / (mem_compressed + 1e-30),
219         "reconstruction_error": mean_error,
220         "memory_dense_mb": mem_dense,
221         "memory_compressed_mb": mem_compressed,
222     }
223
224
225 def trajectory_kinetic_energy_comparison(traj: Trajectory, params: TGVParams) -> dict:
226     """Compare kinetic energy from trajectory vs exact solution."""
227     U, V = traj.to_tensor()
228     ke_num = [kinetic_energy(U[i], V[i], params.rho) for i in range(len(traj.times))]
229     ke_exact = [kinetic_energy(*exact_velocity(traj.X, traj.Y, t, params), params.rho)
230                 for t in traj.times]
231     ke_error = [abs(num - ex) / (abs(ex) + 1e-30) for num, ex in zip(ke_num, ke_exact)]
232     return {
233         "times": traj.times,
234         "ke_numerical": ke_num,
235         "ke_exact": ke_exact,
236         "ke_error": ke_error,
237         "ke_error_mean": float(np.mean(ke_error)),
238         "ke_error_max": float(np.max(ke_error)),
239     }

```